# Deep learning in computer vision - HW#1

Tomáš Belák

November 21, 2025

**Abstract**

## Contents

# 1 Introduction

The aim of this work was to try out different hyperparameters and methods to see what works and what does not work when using a convolutional neural network to classify the CIFAR-10 data set. I tried out different activation functions, optimizers, dropout, data augmentation and analyzed the results compared to a baseline model. After gaining intuition I focused on finding the best model based on acuraccy on the CIFAR-10 test set.

# 2 CIFAR-10 dataset

The CIFAR-10 training dataset contains 50000 RGB images. The images are of size 32x32 pixels. The images are labeled with 1 out of 10 total classes. The dataset contains a separate test set with 10000 images. All classes are equally represented in both the train set and also the test set.

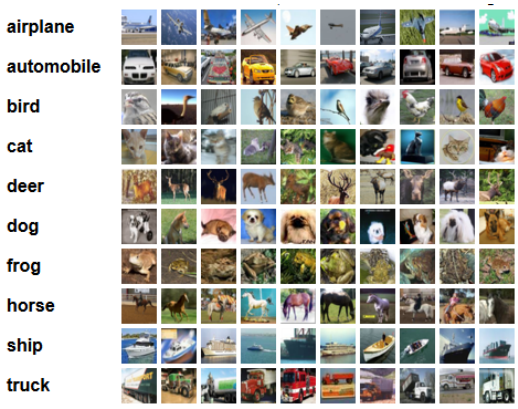



Figure 1: Showcase of the CIFAR-10 dataset.

# 3 Baseline model

The goal for the baseline model was for it to achieve ±70% validation accuracy on the CIFAR-10 dataset as quickly as possible. As a baseline model I implemented a convolutional neural network model. The network consisted of 3 convolutional layers with padding and bias included. After convolution followed max pooling and ReLU activation function, with adaptive average pooling applied before one fully connected layer. The inference was done with the softmax activation function which turns logits into probabilities that sum to 1 across all classes. For training of the baseline model, 90% of

the train set images were used (45000 samples). The rest of the train set was used for validation. The dataset was split into train set and validation set with a fixed random seed which was later used across all experiments. The model was trained using mini-batch training with batch size equal to 32. Cross-entropy was chosen as the loss function as it is suitable for multi-class classification with softmax. For minimizng the loss function the Adam optimizer was selected with a learning rate equal to $10^{-3}$. This model was able to reach the goal of circa 70% validation accuracy in about 13 epochs. I trained the model for 40 epochs.

After the 40 epochs the validation accuracy was at around 73%. The training loss at 0.33 and validation loss at 0.96. As the graph 2 shows, the last validation loss was not the minimal one across training. This pattern, decreasing training loss but increasing validation loss, indicates that the model began to overfit the training data. What is interesting that even if validation accuracy continues to grow the validation loss with cross-entropy can still increase. This happens because the model becomes more confident in its wrong predictions which are penalized more heavily by cross-entropy.

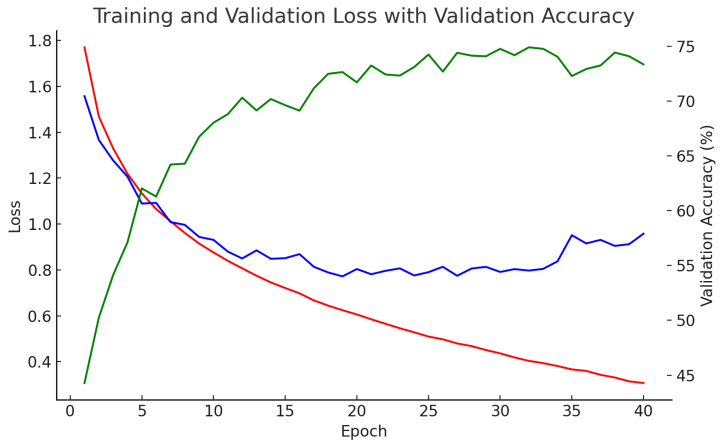



Figure 2: The evolution of the training loss (red), validation loss (blue) and validation accuracy (green) over 40 epochs of training.

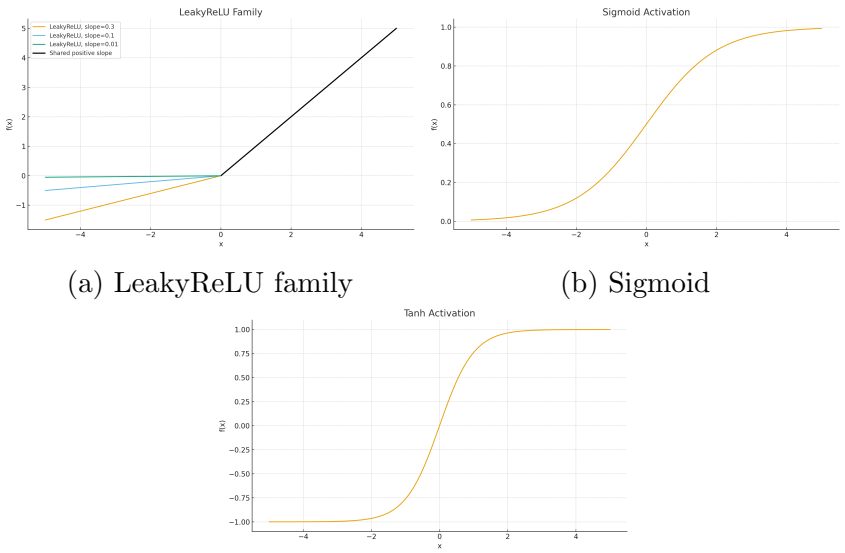
# 4 Experiments

The baseline model began to overfit the train set at around 40 epochs. In this section I applied different activation functions, used different optimizers, batch sizes, utilized drop out and augmenation in order to see which of the mentioned decreases overfitting and therefore enables longer training which promises better results.

## 4.1 Activation functions

The baseline model used ReLU activation function inside. ReLU is defined simply as $ReLU(x) = max(0, x)$. In this part of work I tried out 3 different activation functions but 5 different setups in total:

- LeakyReLU with its default negative slope parameter
- LeakyReLU with negative slope parameter 0.3
- LeakyReLU with negative slope parameter 0.1
- Sigmoid activation function
- Hyperbolic tangent (Tanh) activation function




(a) LeakyReLU family (b) Sigmoid

(c) Tanh

Figure 3: Activation functions used in the model.

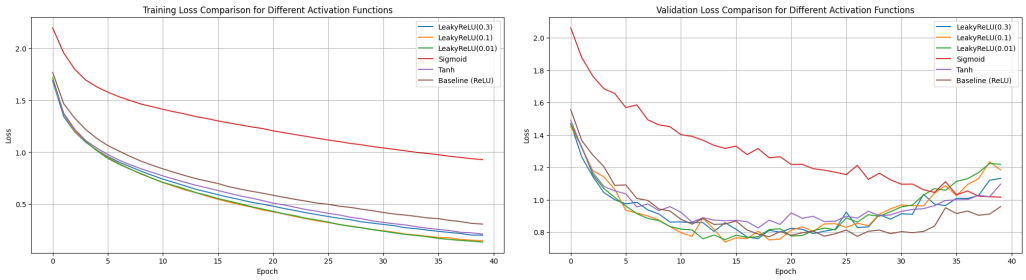
The experiments with different activation functions were trained with the same hyperparameters as the baseline model. In the figure 5 we can see that with using LeakyReLU the training loss converged faster compared to the baseline model, however the models still overfit the training data (validation loss began to rise again). Interestingly using a small negative slope seemed to have the same effect on convergence as using a 10-times steeper slope. As the summary table in the figure n.4 shows, all activations from the LeakyReLU family improved the validation accuracy compared to the baseline. Even

though that both the training and validation loss when using hyperbolic tangens converged to similar values compared to LeakyReLU with negative slope 0.3, the validation accuracy was about 6% worse.

| | Final Train Loss | Final Val Loss | Final Val Accuracy |
|---|---|---|---|
| **Baseline (ReLU)** | 0.307193 | 0.957867 | 0.7336 |
| **LeakyReLU(0.3)** | 0.200458 | 1.132414 | 0.7404 |
| **LeakyReLU(0.1)** | 0.145851 | 1.184102 | 0.7446 |
| **LeakyReLU(0.01)** | 0.133386 | 1.219251 | 0.7448 |
| **Sigmoid** | 0.928576 | 1.016496 | 0.6356 |
| **Tanh** | 0.211364 | 1.097226 | 0.6864 |

Figure 4: Summary table of the trainig of models with different activation functions.

Using the sigmoid activation significantly slowed down the convergence of the training loss. At 40 epochs it seemed that the model was still not overtrained as the trend of the validation loss was still decreasing. However, as the summary table in figure 4 shows, the accuracy was significantly lower after 40 epochs then compared to every other model. Using sigmoid activation could use a longer training to see if the pattern of overfitting would eventually show and what validation accuracy could be reached.




(a) Training loss (b) Validation loss

Figure 5: Comparison of losses during training of models with different validation functions.

## 4.2 Optimization

The baseline model used the Adam optimizer with default settings in training. In this part of work I tried out 5 different optimizer configurations:

- Adam optimizer with learning rate $10^{-4}$ and default $(\beta_1, \beta_2) = (0.9, 0.999)$
- Adam optimizer with learning rate $10^{-2}$ and $(\beta_1, \beta_2) = (0.85, 0.99)$
- Stochastic Gradient Descent (SGD) optimizer with learning rate $10^{-2}$ and momentum 0.9
- Stochastic Gradient Descent (SGD) optimizer with learning rate $10^{-1}$ and momentum 0.9
- RMSprop optimizer with default hyperparameters

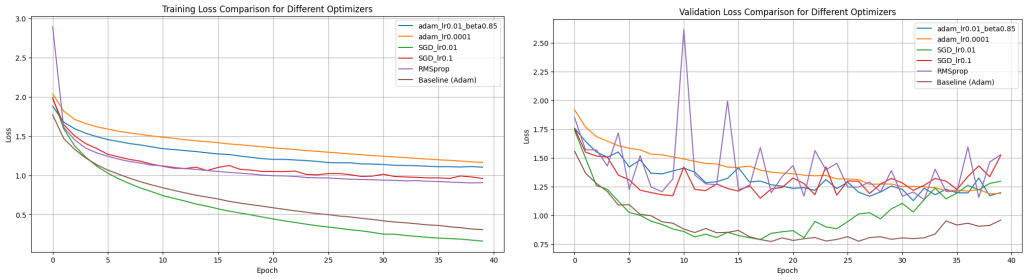
Adam is an optimizer that changes each weight in the network using both the current gradient and a memory of past gradients. It keeps two moving averages: one for the gradients (first moment) and one for the squared gradients (second moment). The parameters $\beta_1$ and $\beta_2$ control how strong this memory is. A larger $\beta_1$ means Adam remembers past gradients for a longer time, so the updates are smoother but react more slowly to new information. The parameter $\beta_2$ controls how much we smooth the squared gradients, which influences how strongly Adam adapts the learning rate for each parameter.

Stochastic gradient descent (SGD) updates the weights by moving them a small step in the opposite direction of the gradient of the loss. With momentum SGD also keeps a part of the previous update called velocity. The momentum value decides how much of this old velocity we keep. A higher momentum should help the optimizer move faster in a stable direction.

RMSprop is another optimizer that automatically adapts the learning rate for each parameter. It maintains a running average of the squared gradients and divides the current gradient by the square root of this average. In this way parameters that often have large gradients receive smaller updates while parameters with small gradients receive relatively larger updates. This should help training stay stable and make learning faster when the gradients are very different across parameters.

The models were trained with the same hyperparameters as the baseline model, other than the optimizer of course. The first configuartion was a default Adam optimizer with lower learning rate, as we can see it slowed down the training convergence significantly compared to the baseline. At epoch 40 there was no sign of overfitting yet. However, the validation accuracy was way worse at this point, as the figure 7 shows. Similar performance showed Adam with default learning rate but slighly lower $\beta_1$ setting. A lower $\beta_1$ setting meant that the optimizer remembered less past gradients. By epoch 40 it seems that this configuration of Adam was choosing a very

small learning rate as the training loss looks like plateuing which means the weights are not getting updated by enough. Similar goes with the default RMSprop configuration, where the training loss shows slow but stable trainig however the generalization on unseen data looks unstable. Using the SGD with learning rate $10^{-2}$ overfit the training data faster than the baseline and the validation accuracies were actually about the same throughout the entire training. SGD with learning rate $10^{-1}$ seemed to be a little too big as even the trainig loss shows some signs of instability. At around epoch 7 it looks like both the validation error loss and validation accuracy just oscillated around a certain value implying no improvement on learning.




(a) Training loss (b) Validation loss

Figure 6: Comparison of losses during training of models with different optimizers.

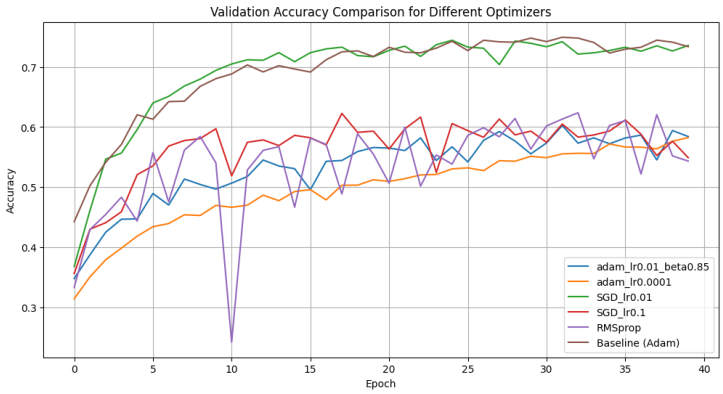
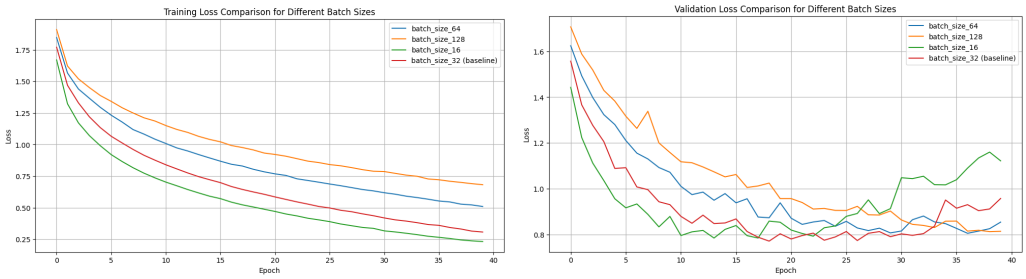



Figure 7: Validation accuracy across the training of models with different optimizers.

## 4.3 Batch size

In mini-batch training method the batch size is another hyperparameter that can influence training. It determines how many samples the model processes before it updates its weights. For this experiment I chose the baseline model. It was retrained with the same hyperparameters other than the batch size which was 32 in the baseline. I tried out 3 different batch sizes - 16, 64 and 128. A small batch size means more frequent updates, while a large batch size makes each update slower but usually more stable which is visible in the figure n.8. The smallest batch size quickly overfit the training data due to frequent updates. Increasing the batch size to 64 seemed to slow down convergence and delay the overfitting but at about the final epoch it seemed to be inevitable to begin overfitting in the next few epochs, if performed. Increasing the batch size to 128 seemed to delay the overfitting even more as at around epoch 40 it still seems to be no signs of it, even though the figure n.9 shows that the validation accuracy is comparable to other configurations that already overfit the training data.




(a) Training loss (b) Validation loss

Figure 8: Comparison of losses during training of models with different batch sizes.

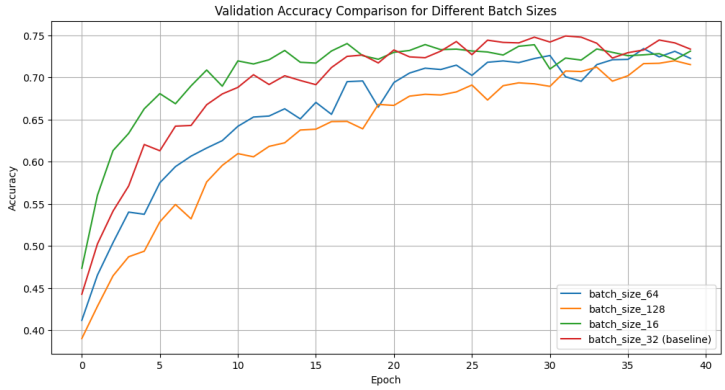



Figure 9: Validation accuracy across the training of models with different batch sizes.

## 4.4 Dropout

In this section, I talk about experimenting with dropout. Dropout is done in order to prevent the neural network from overfitting to the training data. This is achieved by setting, with probability $p$, the output neurons in a layer to zero during training which forces the remaining neurons to learn more robust features that are not dependent on specific neighbors and therefore improve generalization. I experimented with 3 dropout configurations:

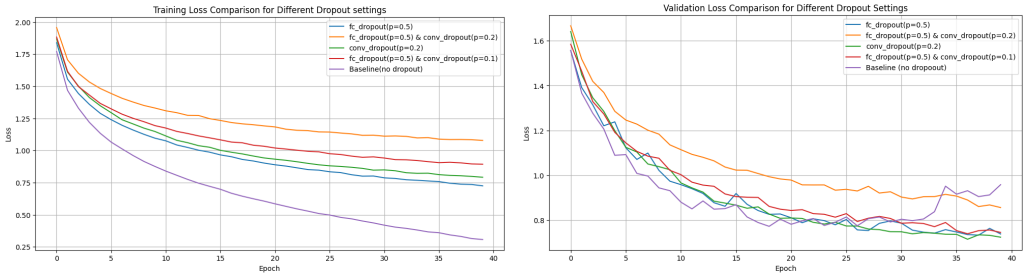
- Dropout in the fully connected layer ($p = 0.5$),
- Dropout in the convolutional layers ($p = 0.2$),
- Dropout in both types of layers $[(p\_fc = 0.5, p\_conv = 0.2) \& (p\_fc = 0.5, p\_conv = 0.1)]$.

First I applied dropout when training on the full training set of 45000 samples (5000 validation samples) and then I tried it out on a training set of just 1000 samples (49000 validation samples). For all trainings the baseline model architecture (apart from the added dropout) and hyperparameters used were the same as the ones used to train the baseline model, unless stated otherwise.

### 4.4.1 Full training set

From the comparison of training and validation losses in the figure n.10 we can see that when training on the full training set of 45000 samples, the dropout technique seemed to help quite a bit. All dropout configurations avoided overfitting the training set during the 40 epochs of training. However,

as graph n.11 shows, applying dropout to the fully connected layer with $p = 0.5$ and $p = 0.2$ to the convolutional layers seemed to need more epochs in order to reach validation accuracy similar to all other configurations including baseline.




(a) Training loss (b) Validation loss

Figure 10: Comparison of losses during training of models with different dropout configurations.

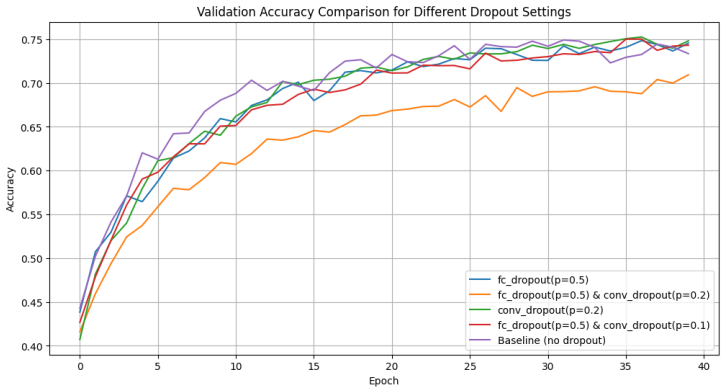
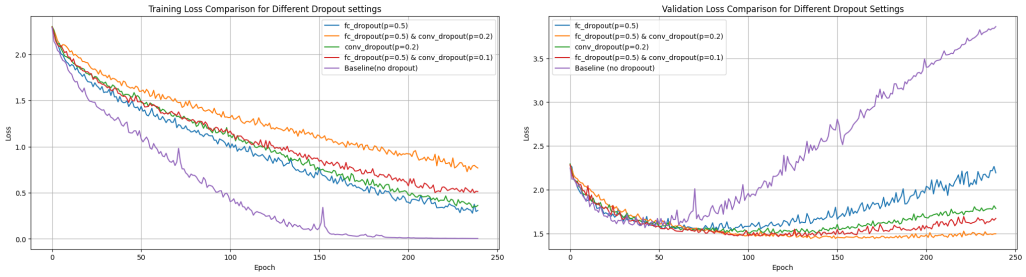



Figure 11: Validation accuracy across the training of models with different dropout configurations.

### 4.4.2 Small train set

When training on the reduced training set of just 1000 samples, the plots in the figure n.12 show that the dropout technique seemed to help quite a bit. All dropout configurations delayed and slowed down the overfitting of the training set compared to the completely overfit baseline. As per figure n.13, applying dropout slightly increased the validation accuracy.




(a) Training loss (b) Validation loss

Figure 12: Comparison of losses during training of models with different dropout configurations.

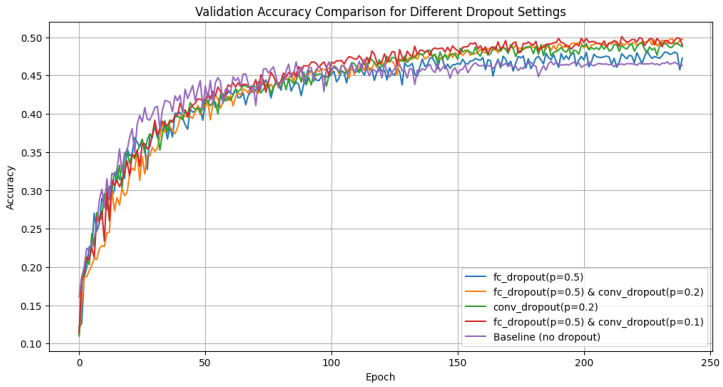



Figure 13: Validation accuracy across the training of models with different dropout configurations.

## 4.5 Augmentation

In this section, I talk about experimenting with data augmentation. Data augmentation is done in order to prevent the neural network from overfitting to the training data, especially when the available dataset is small. Data augmentation means applying various transformations (like rotation, scaling, shifting or flipping) to the existing training data. This process increases the diversity of the data seen by the model, which forces the model to learn robust features which improves generalization. I experimented with 3 different levels of data augmentation:

- **Configuration: Easy**

  `transforms.RandomCrop(32, padding=4)`**:** Randomly crops the image to $32 \times 32$ pixels after padding the edges by 4 pixels.

`transforms.RandomHorizontalFlip(p=0.5)`: Flips the image horizontally with a probability $p = 0.5$.

- **Configuration: Medium**

  `transforms.RandomResizedCrop(32, scale=(0.8, 1.0))`: Crops the image to a random size and aspect ratio, then scales it to $32 \times 32$ pixels. The area scale factor is randomly chosen from $[0.8, 1.0]$.

  `transforms.RandomHorizontalFlip(p=0.5)`: Flips the image horizontally with a probability $p = 0.5$.

  `transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3)`: Randomly changes the brightness, contrast, and saturation of the image by factors up to 0.3.

- **Configuration: Final boss**

  `transforms.RandomAffine(degrees=20, translate=(0.1, 0.1), scale=(0.9, 1.1), shear=10)`: Applies random affine transformation, including rotation (up to $\pm 20°$), translation (up to $\pm 10\%$ horizontally and vertically), scaling (by a factor in $[0.9, 1.1]$), and shearing (up to $\pm 10°$).

  `transforms.RandomPerspective(distortion_scale=0.4, p=0.5)`: Applies random perspective transformation with a distortion scale of 0.4 and probability $p = 0.5$.

  `transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4)`: Randomly changes brightness, contrast, and saturation by factors up to 0.4.

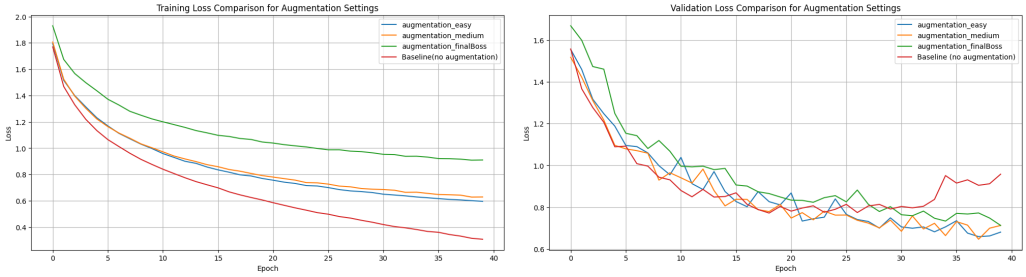
  `transforms.RandomHorizontalFlip(p=0.5)`: Flips the image horizontally with a probability $p = 0.5$.

First I applied data augmentation when training on the full training set of 45000 samples (5000 validation samples) and then I tried it out on a training set of just 1000 samples (49000 validation samples). For all trainings the baseline model architecture (apart from the added dropout) and hyperparameters was used, unless stated otherwise.

### 4.5.1 Full training set

The three data augmentation levels really helped stop overfitting when we trained on the whole trainig set for 40 epochs. The random transformations

done to the images in each batch made it way harder for the model to just memorize the training data. This is visible from the training loss plot in figure n.14a as the training converged more slowly compared to the baseline without data augmentation. Also the validation loss plot shows zero overfitting while the models even got slightly better validation accuracy scores than the baseline.




(a) Training loss (b) Validation loss

Figure 14: Comparison of losses during training of models with different data augmentations.

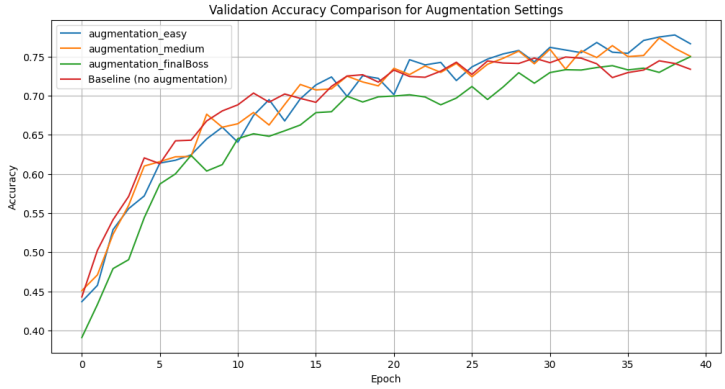
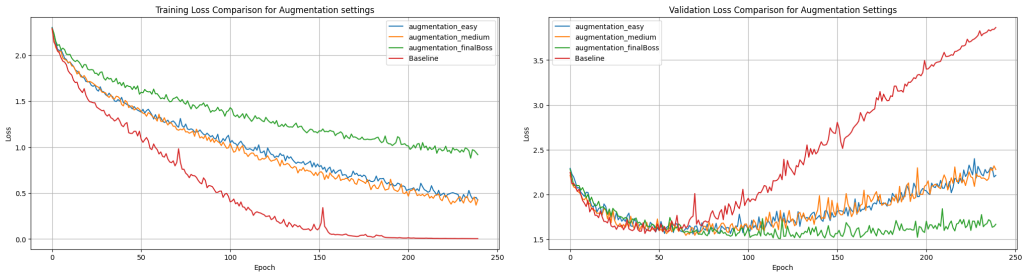



Figure 15: Validation accuracy across the training of models with different data augmentations.

### 4.5.2 Small train set

When training on a set with just 1000 samples, I increased the number of epochs from 40 to 240. From the training loss plot in the fiugre n.12a we can see that the baseline model memorized the training data as the final training loss was about 0.003. From the validation loss plot we can see that for the easy and medium levels of augmentation overfitting appeared about the same

time as with baseline, around the 100th epoch, but the increase of validation loss was slower. Only the aggresive (finalBoss) augmentation helped to delay overfitting as it started appearing at around epoch 170. In the figure n.17, the validation accuracies were about the same for each level of augmentation with almost no improvement to the overfit baseline.




(a) Training loss (b) Validation loss

Figure 16: Comparison of losses during training of models with different data augmentations.

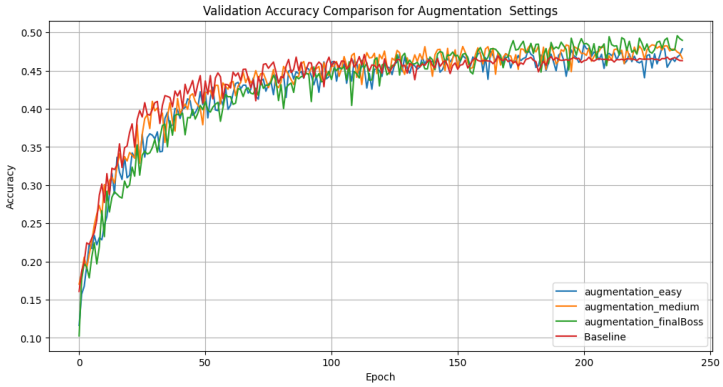



Figure 17: Validation accuracy across the training of models with different data augmentations.

## 4.6 Deep web

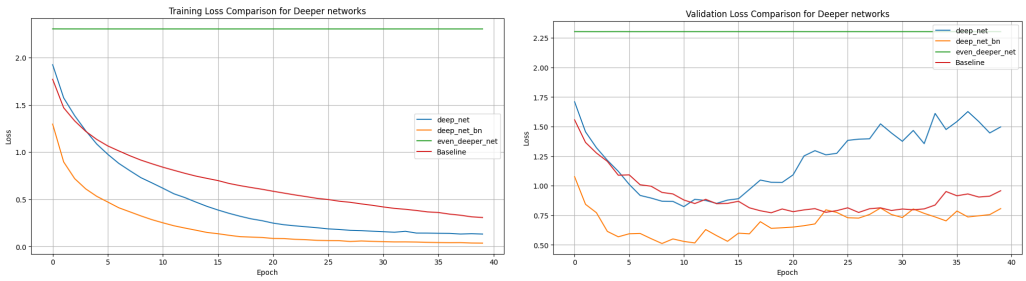
In this section, I experimented with deeper model architectures and utilizing batch normalization within, in order to fight overfitting. Batch normalization is done by taking the mean and variance for every batch of training data to normalize the inputs so the values do not get too widely spread out. This should stop internal data distribution from shifting around too much,

which should speed up the training and make the model more stable. I experimented with these configurations/architectures:

- architecture with 9 convolutional layers and 2 fully connected layers (deep_net),
- the same architecture as above but with batch normalization (deep_net_bn),
- architecture with 12 convolutional layers and 3 fully connected layers (even_deeper_net).

Apart from the architecture, all hyperparameters used to train these models were the same as the ones used for the training of the baseline model.

From the loss plots in the figure n.18] we can see that the deep_net and deep_net_bn architectures learned the training data faster than the baseline. However, even with batch normalization the pattern of overfitting the training data training loss, decreasing and validation loss decreasing and then increasing, appeared at about the same epoch, but the rise of validation loss was significantly slower. As per the figure n.19, the model with batch normalization achieved significantly better validation accuracy compared to all models trained during these experiments, 84.58%. The even_deeper_net architecture with 15 layers was disappointing and learned nothing possibly because of the gradient vanishing during backpropagation. Evidently, networks of this depth really need the stabilization mechanisms like batch normalization or residual connections to maitnain the flow of gradients.




(a) Training loss (b) Validation loss

Figure 18: Comparison of losses during training of deeper models.

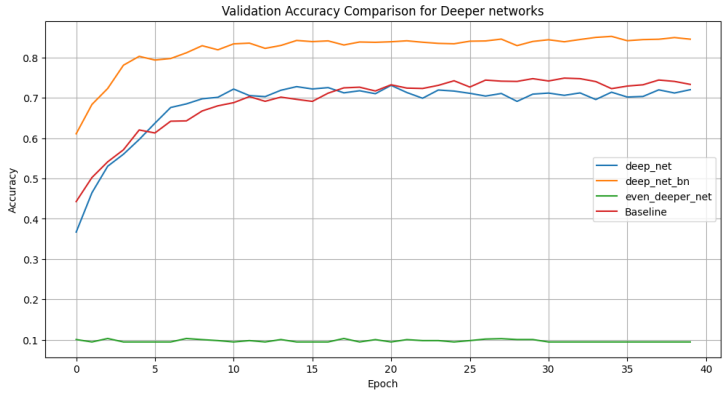



Figure 19: Validation accuracy across the training of deeper models.

# 5 Best model

For the best model configuration I decided to combine what seemed to work from each experiment. For the model architecture I chose the deep_net_bn architecture from section 4.6. This architecture has 9 convolutional layers and 2 fully connected layers and utilizes batch normalization. For optimizer, I went with the pyTorch default Adam optimizer setting. The batch size was set to 64. I also utilized the data augmentation with the medium level transform configuration from section 4.5. Dropout was not applied (I forgot to add it). The model was trained for 120 epochs, however after each epoch checkpoint models were created. I decided to evaluate the model that reached the minimal validation loss (0.354, epoch 30) and the model that reached the maximal validation accuracy (0.9014, epoch 96). As the table n.1 shows, the best performing model was model n.3 that was chosen based on maximum validation accuracy. However, looking at the plot of the losses during training in the figure n.20, we can see that before the corresponding epoch 96 the models had already begin to overfit the training data but not dramatically.

| Model | Selection criterion | Test accuracy (%) |
|---|---|---|
| 1 | Baseline | 72.06 |
| 2 | Minimum validation loss | 88.41 |
| 3 | Maximum validation accuracy | 89.26 |

Table 1: Comparison of test set accuracies for different model selection strategies.

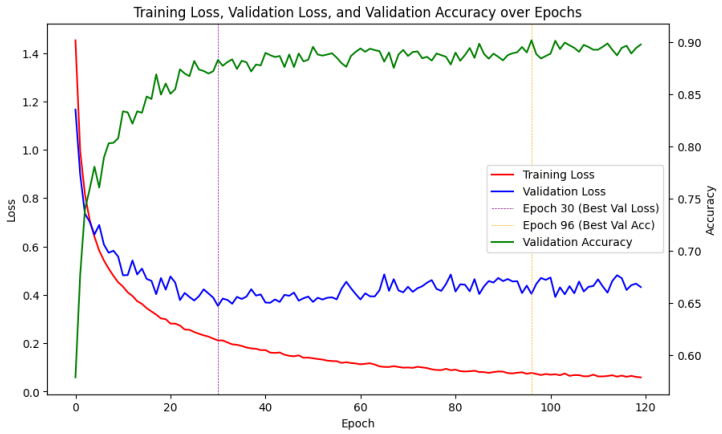



Figure 20: Training loss, validation loss and validation accuracy across the training of the best model.

# 6 Conclusion

To wrap things up, this assignment explored how various hyperparameters impact a CNN's performance on the CIFAR-10 dataset. Regarding activation functions, the LeakyReLU family outperformed the others, whereas Sigmoid and Tanh were just too slow or ineffective. Adam with default pyTorch settings turned out to be the most reliable optimizer, beating the unstable or slow results I got from SGD (high learning rate) and RMSprop. I also noticed that using a larger batch size of 64 or 128 successfully delayed overfitting. Dropout proved to be a solid method for preventing overfitting on the full dataset, and it even helped slightly when training on limited data. Data augmentation was essential for good generalization, though on the small dataset, only the aggressive transformations made a real difference. Moving to the architecture, the 9-layer network with batch normalization performed the best, while the even deeper 15-layer model failed completely possibly due to vanishing gradients. Ultimately, by combining the most effective settings, the final model achieved a test accuracy of 89.26%.